

갠트리 펀칭 시스템 통신/제어 사양서

본 문서는 HMI(C#) 개발자와 펌웨어(C++) 개발자가 독립적으로 작업할 수 있도록 작성된 인터페이스 사양서입니다. 양측은 본 문서의 **5장(인터페이스 합의 사항)** 을 공통 계약으로 간주하고 진행합니다.

1. 시스템 개요

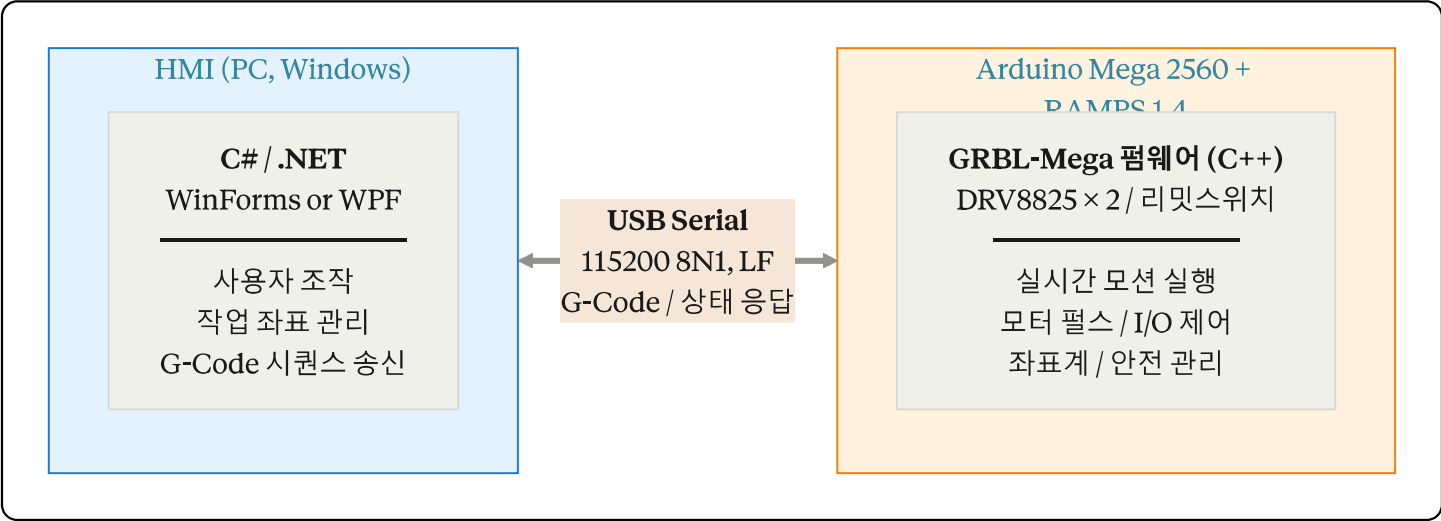
1.1 하드웨어 구성

구분	사양
MCU 보드	Arduino Mega 2560
모션 쉴드	RAMPS 1.4
스텝 드라이버	DRV8825 (X축, Y축)
모션 축	2축 갠트리 (X, Y)
액추에이터	펀칭 메커니즘 1개 (솔레노이드/에어실린더 가정)
호스트 ↔ MCU	USB (Virtual COM Port)

1.2 동작 시나리오

갠트리 구조의 X-Y 축이 지정된 좌표로 이동 → 해당 위치에서 액추에이터로 펀칭 → 다음 좌표로 이동을 반복 → 작업 완료 후 원점 복귀.

1.3 시스템 아키텍처



2. 업무 분담

2.1 C++ (Arduino) 담당 범위

항목	내용
펌웨어 베이스	GRBL-Mega (또는 <code>grbl-Mega-5X</code>) 포팅
핀 매핑	RAMPS 1.4 ↔ GRBL <code>cpu_map.h</code> 수정
모션 파라미터	스텝/mm, 최대 속도, 가감속, 소프트/하드 리미트 캘리브레이션
홈잉	리미트 스위치 배선 + GRBL 홈잉 설정 (<code>\$22~\$27</code>)
펀칭 I/O	스핀들 핀(M3/M5) 또는 디지털 출력을 액추에이터 제어로 전환
테스트	단독 시리얼 콘솔에서 G-Code 명령으로 동작 검증

C++ 측은 G-Code 파싱, 펄스 생성, 가감속, 좌표계 관리, 통신 프로토콜을 직접 구현하지 않습니다. GRBL이 모두 제공합니다.

2.2 C# (HMI) 담당 범위

항목	내용
시리얼 통신	<code>System.IO.Ports.SerialPort</code> , 115200 8N1, LF 종단
송신 큐	G-Code 라인 큐 + 페이싱(Send-Response 또는 Character-Counting)
상태 폴링	별도 스레드에서 <code>?</code> 주기적 전송 → <code><...></code> 파싱
좌표/시퀀스 관리	펀칭 좌표 리스트 → G-Code 시퀀스 변환
UI	작업 화면, Jog 컨트롤, 알람/에러 표시, 진행률, E-Stop
안전 처리	Alarm 상태 감지, 사용자 확인 후 홈잉 유도, 일시정지/재개/리셋

2.3 책임 경계

- C++ 은 "한 줄의 G-Code를 받아서 정확히 실행하는 것"까지 책임진다.
- C# 은 "사용자 의도를 G-Code 라인 시퀀스로 변환하여 정확한 순서·타이밍으로 흘려보내는 것"까지 책임진다.

- 공통 계약은 5장의 G-Code/M-Code 목록 + GRBL (\$) 설정값.

3. 통신 사양

3.1 물리/링크 계층

항목	값
인터페이스	USB CDC (Virtual COM Port)
Baud rate	115200
데이터 비트	8
패리티	None
스톱 비트	1
흐름 제어	None
라인 종단	LF ($\backslash n$, 0x0A)
인코딩	ASCII

3.2 프로토콜 계층 — GRBL G-Code 프로토콜

모든 통신은 텍스트 기반 라인 단위입니다.

3.2.1 명령 종류

분류	형태	응답	설명
G-Code/M-Code	$G0\ X100\ Y50\backslash n$	ok 또는 $error:N$	모션/I/O 명령. 파서가 받아 플래너 큐에 적재
시스템 명령 ($\$$)	$\$H\backslash n, \$\$ \backslash n$	ok 또는 결과 후 ok	홈, 설정 조회/변경 등
실시간 명령 (1바이트)	$?, !, \sim, \backslash x18$	즉시 응답 또는 동작 변경	라인 종단 불필요, 큐 무시하고 즉시 처리

3.2.2 실시간 명령

바이트	의미
[?] (0x3F)	상태 보고 요청 → <Idle MPos:0.000,0.000,0.000 FS:0,0>
[!] (0x21)	Feed Hold (감속 정지)
[~] (0x7E)	Cycle Start / Resume
[\x18] (Ctrl-X)	Soft Reset (모든 큐 비움, 위치 유지)

실시간 명령은 다른 라인 송신 중간에 끼어들어도 OK입니다. 큐와 무관하게 즉시 처리됩니다.

3.2.3 **[ok]**의 의미 — 매우 중요

[ok]는 "라인을 파싱해서 플래너 버퍼에 적재 완료"를 의미합니다. **모터가 그 위치에 도달했다는 의미가 아닙니다.**

- 모션 완료 여부는 **[?]** 폴링으로 상태가 **[Idle]**인지 확인해야 알 수 있다.
- 짧은 라인을 빠르게 보내면 **[ok]**는 즉시 오지만 실제 모션은 한참 뒤에 끝난다.

3.2.4 상태 응답 형식

```
<State|MPos:X,Y,Z|FS:Feed,Spindle|WC0:X,Y,Z>
```

State 값	의미
[Idle]	큐 비어있고 정지 상태. 작업 가능
[Run]	모션 실행 중
[Hold:0] / [Hold:1]	Feed Hold 정지 / 감속 중
[Jog]	Jog 명령 실행 중
[Alarm]	알람 상태. 모션 명령 거부됨
[Door] / [Check] / [Home] / [Sleep]	기타 상태

3.2.5 에러/알람 응답

error:N ← G-Code 파싱 또는 실행 단계의 에러. N = 에러 코드 번호
ALARM:N ← 안전 관련 알람. 모션 정지 + 잠금

에러/알람 코드 목록은 GRBL 공식 문서 ([error_codes.md](#)) 참조. 자주 만나는 코드:

- [error:9](#) — Locked, \$H 필요
- [error:20](#) — Unsupported command
- [ALARM:1](#) — 하드 리밋 트리거
- [ALARM:2](#) — 소프트 리밋 초과

3.3 송신 페이싱 전략

3.3.1 Simple Send-Response (기본)

```
HOST → "G0 X10\n"  
HOST ← "ok\n"  
HOST → "G0 X20\n"  
HOST ← "ok\n"
```

매 라인마다 [ok](#) 대기. 구현 단순. 펀칭 작업처럼 라인 사이에 짧은 간격이 있어도 무방한 워크로드에 적합.

3.3.2 Character-Counting Streaming (성능)

GRBL의 RX 버퍼 128바이트가 비는 만큼 미리 채워서 보냄. 짧은 세그먼트가 많은 곡선 절삭 등에 필요. 본 시스템(펀칭)에서는 불필요하므로 1차 구현 시 채택하지 않음.

4. 운영 시퀀스

4.1 부팅 시퀀스

```
sequenceDiagram  
    participant U as 사용자  
    participant H as HMI (C#)  
    participant G as GRBL (Arduino)  
  
    H->>G: SerialPort.Open(COMx, 115200)  
    G-->>H: "Grbl 1.1h ['$' for help]"  
    H->>G: "?"  
    G-->>H: <Alarm|MPos:0,0,0|...>  
    Note over H: 홈잉 ON 설정 시<br/>부팅 직후는 Alarm 상태  
    H->>U: "기계 원점 복귀 필요" 안내<br/>[HOME] 버튼 활성화
```

```
U->>H: [HOME] 클릭
H->>G: "$H"
Note over G: 홈잉 수행<br/>(수 초 소요, 블로킹)
G-->>H: "ok"
H->>G: "?"
G-->>H: &lt;Idle | ...&gt;;
Note over H: 작업 가능 상태로<br/>UI 전환
```

4.2 작업 시퀀스 (펀칭 N회)

gcode

```
G90                ; 절대좌표 모드
G21                ; mm 단위
G0 X100 Y50 F3000  ; 1번 펀칭 위치로 급속이동
M3                ; 액추에이터 ON
G4 P0.5            ; 0.5초 유지 (펀칭 시간)
M5                ; 액추에이터 OFF
G0 X200 Y80
M3
G4 P0.5
M5
...
G0 X0 Y0           ; 원점 복귀
```

C# 측은 위 시퀀스를 라인 단위로 큐에 넣고 페이싱하며 송신. 마지막 라인 **ok** 수신 후 **?** 폴링으로 **Idle** 확인되면 작업 완료로 판단.

스트리밍 동작 — **ok**와 실제 모션이 분리됨에 주의:

```
sequenceDiagram
    participant H as HMI (C#)
    participant P as GRBL Parser/Planner
    participant M as 모터/액추에이터

    H->>P: G0 X100 Y50
    P-->>H: ok (큐 적재 완료)
    H->>P: M3
    P-->>H: ok
    H->>P: G4 P0.5
    P-->>H: ok
    H->>P: M5
    P-->>H: ok
    Note over H: 호스트 송신 완료<br/>이후엔 ? 폴링만
    P->>M: 펄스 생성 → 1번 위치로 이동
    M-->>P: 도달
    P->>M: 액추에이터 ON
    Note over M: 0.5초 dwell
    P->>M: 액추에이터 OFF
    Note over P,M: 큐 비울 때까지<br/>자동 진행
    H->>P: ?
    P-->>H: &lt;Idle | MPos:... | ...&gt;;
    Note over H: 작업 완료 판정
```

4.3 일시정지/재개/취소

사용자 동작	C# 송신	결과
일시정지	 (1바이트)	감속 정지, 위치/큐 유지
재개	 (1바이트)	정지 지점부터 큐 이어서 실행
취소(E-Stop 소프트)	 (\x18) (Ctrl-X)	즉시 리셋, 큐 비움, 위치는 유지
비상정지(하드)	별도 물리 버튼 → 전원 차단 권장	위치 손실, 재 홈잉 필요

4.4 알람 발생 시

```
flowchart LR
    A[GRBL이 ALARM:N 송신] --> B[모든 후속<br/>모션 명령 거부]
    B --> C[C# 측<br/>알람 모달 표시<br/>원인 안내]
    C --> D[사용자<br/>[Reset & Home]<br/>클릭]
    D --> E[Yes]
    E --> F["Ctrl-X (\x18) 송신"]
    F --> G[Idle 복귀]
```

```
style A fill:#ffcdd2,stroke:#c62828
style G fill:#c8e6c9,stroke:#2e7d32
```

5. 인터페이스 합의 사항 (양측 공통 계약)

이 장은 양측이 반드시 합의하고 동결해야 하는 항목입니다.

5.1 사용 G-Code/M-Code 목록

코드	용도	비고
<code>G0 X.. Y.. F..</code>	급속 이동	편칭 위치 이동에 사용
<code>G1 X.. Y.. F..</code>	직선 이동 (선택)	본 시스템에서는 G0와 동일 동작
<code>G4 P<sec></code>	Dwell (대기)	편칭 유지 시간
<code>G90</code> / <code>G91</code>	절대/상대좌표	일반 작업은 G90
<code>G21</code>	mm 단위	시작 시 1회
<code>G92 X0 Y0</code>	작업 원점 설정	필요시 사용
<code>M3</code>	편칭 액추에이터 ON	GRBL 스피들 ON 핀 재사용
<code>M5</code>	편칭 액추에이터 OFF	GRBL 스피들 OFF 핀 재사용
<code>\$H</code>	홈잉	부팅 후 필수
<code>\$J=...</code>	Jog	Jog UI에서 사용
<code>\$</code>	설정 조회	진단용
<code>\$X</code>	알람 해제	디버깅용 (운영에서는 비권장)

5.2 좌표계 기준점

- 기계 원점 (Machine Zero):** 홈잉 후 결정. 갠트리의 좌측 전방 모서리 또는 우측 후방 모서리(설치에 따름).
- 작업 원점 (Work Zero):** 편칭 도면의 (0,0) 위치. `G54` 또는 `G92`로 설정.
- 편칭 좌표는 작업 원점 기준으로 도면에 정의하고, C# 측은 이를 그대로 G-Code로 변환한다.

5.3 핀 매핑 합의

기능	RAMPS 1.4 단자	비고
X 스텝/방향/인에이블	X-MOT (X_STEP, X_DIR, X_EN)	DRV8825
Y 스텝/방향/인에이블	Y-MOT	DRV8825
X 리밋	X-MIN 또는 X-MAX	NC/NO 합의 필요
Y 리밋	Y-MIN 또는 Y-MAX	NC/NO 합의 필요
편칭 액추에이터	RAMPS의 "Heated Bed" 또는 D9/D10 핀	GRBL의 SpindleEnable 핀으로 매핑

5.4 GRBL 설정값 (§) — C++ 측이 캘리브레이션 후 확정

설정	의미	초기 제안값
§0	스텝 펄스 시간 (μs)	10
§2	스텝 포트 invert	0 (DRV8825 기준)
§3	방향 포트 invert	축 방향에 따라
§5	리밋 핀 invert	스위치 NO/NC에 따라
§20	소프트 리밋	1
§21	하드 리밋	1
§22	홈잉 활성화	1
§23	홈잉 방향 마스크	설치 방향 따라 (3 = X,Y 음의 방향)
§24	홈잉 feed (mm/min)	25
§25	홈잉 seek (mm/min)	500
§27	pull-off (mm)	1.0
§100	X 스텝/mm	DRV8825 마이크로스텝/벨트 피치 따라
§101	Y 스텝/mm	//
§110) / (§111)	X/Y 최대속도 (mm/min)	캘리브레이션

설정	의미	초기 제안값
<code>\$120</code> / <code>\$121</code>	X/Y 가속도 (mm/sec ²)	캘리브레이션
<code>\$130</code> / <code>\$131</code>	X/Y 작업 영역 (mm)	갠트리 실측

확정된 설정값은 본 문서 부록 또는 별도 `grbl_config.txt` 로 형상 관리한다.

6. C++ 측 상세 가이드

6.1 GRBL이란

GRBL은 Arduino용 오픈소스 G-Code 인터프리터/모션 컨트롤러 펌웨어입니다. 라이브러리가 아니라 펌웨어(통째로 올리는 프로그램)이며, 다음 기능을 모두 제공합니다.

- G-Code 파서
- 모션 플래너 (가감속, look-ahead)
- 스텝 펄스 생성 (타이머 인터럽트 기반, 30 kHz 이상)
- 좌표계 관리 (G54~G59, G92, MPos/WPos)
- 홈잉, 소프트/하드 리밋
- USB 시리얼 프로토콜 (`ok`), `error`, 실시간 명령, `?` 상태 응답)
- 스피들/쿨런트 I/O 제어 (M3~M9)

원본 GRBL은 ATmega328 (Uno) 대상이라 본 시스템에서는 **Mega 2560 포팅 버전**을 사용한다.

저장소	URL	비고
grbl-Mega	<code>https://github.com/gnea/grbl-Mega</code>	Mega 2560 공식 포트
grbl-Mega-5X	<code>https://github.com/fra589/grbl-Mega-5X</code>	5축 확장판 (2축에도 사용 가능)

본 시스템은 2축이므로 `grbl-Mega` 를 권장한다.

6.2 작업 절차

Step 1. 저장소 클론 및 IDE 준비

```
bash
git clone https://github.com/gnea/grbl-Mega.git
```

Arduino IDE에서 **Sketch → Include Library → Add .ZIP Library** 로 `grbl` 폴더를 추가하거나, 클론 디렉터리를 `Arduino/libraries/grbl-Mega` 로 심볼릭 링크.

Step 2. 핀 매핑 — `cpu_map.h` 수정

GRBL은 RAMPS 1.4를 기본 지원하지 않으므로 `grbl/cpu_map.h` 에 RAMPS 매핑을 정의해야 한다. 기존 코드에 다음 블록을 추가하거나, `defaults.h` 에서 `DEFAULTS_RAMPS_BOARD` 같은 식별자를 정의해 분기시킨다.

cpp

```
// grbl/cpu_map.h 의 적절한 #ifdef 분기 안에 추가
#ifdef CPU_MAP_RAMPS_BOARD

// 시스템 인터럽트 (변경 없음)
#define LIMIT_INT          PCIE0
#define LIMIT_INT_vect     PCINT0_vect

// X 축 - RAMPS X 모터 단자
#define X_STEP_BIT         0    // PORTA0 = D54 = AUX X_STEP
#define X_DIRECTION_BIT    1    // PORTA1 = D55 = AUX X_DIR
#define X_ENABLE_BIT       7    // PORTA7 = D38 = AUX X_EN

// Y 축 - RAMPS Y 모터 단자
#define Y_STEP_BIT         6    // PORTF6 = D60
#define Y_DIRECTION_BIT    7    // PORTF7 = D61
// ... (RAMPS 1.4 핀맵 표 참조하여 채움)

// 리밋 스위치 - RAMPS X-MIN, Y-MIN
#define X_LIMIT_BIT        5    // D3
#define Y_LIMIT_BIT        6    // D14
// 풀업 사용 (스위치 NO 기준)

// 펀칭 액추에이터 - 스피들 ENABLE 핀을 RAMPS D9 (FAN) 또는 D10 (HEATER) 로 매핑
#define SPINDLE_ENABLE_BIT  6    // 예: PORTH6 = D9
#define SPINDLE_DIRECTION_BIT 5 // 사용 안 함이지만 정의 필요

#endif
```

주의: 위 비트 번호는 예시이며, RAMPS 1.4 ↔ Mega 2560 핀맵 표를 보고 정확한 PORT/BIT를 확인해야 한다. 참고 자료: https://reprap.org/wiki/RAMPS_1.4 의 "Pin Assignments".

선택한 매핑을 활성화하려면 `config.h` 상단에서:

cpp

```
// config.h
// #define DEFAULTS_GENERIC
#define DEFAULTS_RAMPS_BOARD
#define CPU_MAP_RAMPS_BOARD
```

Step 3. 기본값 — defaults.h

홈 방향, 작업 영역, 스텝/mm 등의 컴파일 타임 기본값을 본 시스템에 맞게 정의한다. 다만 이 값들은 EEPROM에 저장되는 (\$) 설정으로 런타임에 덮어쓸 수 있으므로, 컴파일 타임에는 안전한 디폴트만 두고 실측 캘리브레이션은 시리얼 콘솔로 \$100=80.0 같은 식으로 한다.

```
cpp

// defaults.h
#ifdef DEFAULTS_RAMPS_BOARD
    #define DEFAULT_X_STEPS_PER_MM      80.0
    #define DEFAULT_Y_STEPS_PER_MM      80.0
    #define DEFAULT_X_MAX_RATE          5000.0    // mm/min
    #define DEFAULT_Y_MAX_RATE          5000.0
    #define DEFAULT_X_ACCELERATION      (500.0 * 60 * 60)    // mm/min^2
    #define DEFAULT_Y_ACCELERATION      (500.0 * 60 * 60)
    #define DEFAULT_X_MAX_TRAVEL        400.0    // mm - 갠트리 실측
    #define DEFAULT_Y_MAX_TRAVEL        400.0
    #define DEFAULT_HOMING_ENABLE        1
    #define DEFAULT_HOMING_DIR_MASK     3        // X,Y 음방향
    #define DEFAULT_HOMING_FEED_RATE    25.0
    #define DEFAULT_HOMING_SEEK_RATE    500.0
    #define DEFAULT_HOMING_PULLOFF      1.0
    #define DEFAULT_HARD_LIMIT_ENABLE   1
    #define DEFAULT_SOFT_LIMIT_ENABLE   1
#endif
```

Step 4. 빌드 및 업로드

Arduino IDE:

1. Tools → Board → **Arduino Mega or Mega 2560**
2. Tools → Port → 해당 COM 포트
3. `grbl-Mega/grblUpload/grblUpload.ino` 열기
4. Upload

업로드 후 시리얼 모니터(115200, "Newline") 열면:

Grbl 1.1h ['\$' for help]

Step 5. 캘리브레이션

시리얼 모니터에서 직접 명령:

\$	\$	← 현재 설정 모두 출력
\$100=80.0	← X 스텝/mm 변경	
\$110=4000	← X 최대속도 변경	
\$130=350	← X 작업 영역 변경	
\$H	← 홈잉 동작 확인	
G91 G0 X10	← 상대 10mm 이동 - 실제로 10mm인지 자로 측정	

스텝/mm 캘리브레이션은 **명령 거리 ÷ 실측 거리 × 현재 스텝/mm** 공식으로 정확히 맞춘다.

Step 6. 펀칭 동작 검증

M3	← 액추에이터 ON 확인 (테스터/소리/시각)
M5	← OFF 확인
G4 P0.5	← Dwell 동작 확인
M3	
G4 P0.5	
M5	

Step 7. 통합 테스트

```
$H
G90 G21
G0 X100 Y50 F3000
M3
G4 P0.5
M5
G0 X0 Y0
```

각 라인마다 **ok** 응답 + **?**로 상태 확인.

6.3 액추에이터 매핑 상세

GRBL의 스피들 제어는 다음 두 명령으로 동작한다.

명령	동작
M3 S<n>	SpindleEnable 핀 HIGH, PWM이 있으면 S값에 따라 듀티
M5	SpindleEnable 핀 LOW

본 시스템에서는 PWM이 필요 없는 단순 ON/OFF이므로 **M3** 만 보내면 된다 (S값 생략 가능).
config.h 에서 가변 PWM을 비활성화해도 좋다:

```
cpp
// config.h
// 스피ن들을 단순 디지털 ON/OFF로 사용
#define DISABLE_LASER_DURING_HOLD
// PWM 가변속이 필요없으면 다음을 정의 (선택)
// #define SPINDLE_ENABLE_OFF_WITH_ZERO_SPEED
```

중요: **M3**/**M5** 는 모션 큐에 들어가므로 자연스럽게 모션과 동기화된다. 즉:

```
G0 X100 Y50    ← 이 모션이 끝난 후
M3             ← 액추에이터 ON
```

순서가 보장된다. 다만 정밀하게 "정지가 완전히 끝난 후 ON"을 보장하려면 **M3** 앞에 **G4 P0**을 끼우는 것이 안전하다.

6.4 안전 관련 설정

- **\$21=1** (하드 리밋) — 운전 중 리밋 스위치가 hit되면 즉시 ALARM:1, 펄스 정지.
- **\$20=1** (소프트 리밋) — 작업 영역(**\$130**/**\$131**) 밖으로의 G-Code는 거부.
- **\$22=1** (홈잉) — 부팅 직후 Alarm 진입, **\$H** 전까지 모션 거부.

위 3개는 모두 활성화한다. 디버깅 중 일시적으로 풀어야 할 때만 **\$X**로 알람 해제.

6.5 소스 수정이 필요한 영역 요약

파일	수정 내용
grbl/cpu_map.h	RAMPS 1.4 핀 매핑 블록 추가
grbl/defaults.h	본 시스템 기본값 블록 추가
grbl/config.h	DEFAULTS_RAMPS_BOARD / CPU_MAP_RAMPS_BOARD 활성화, 옵션 조정

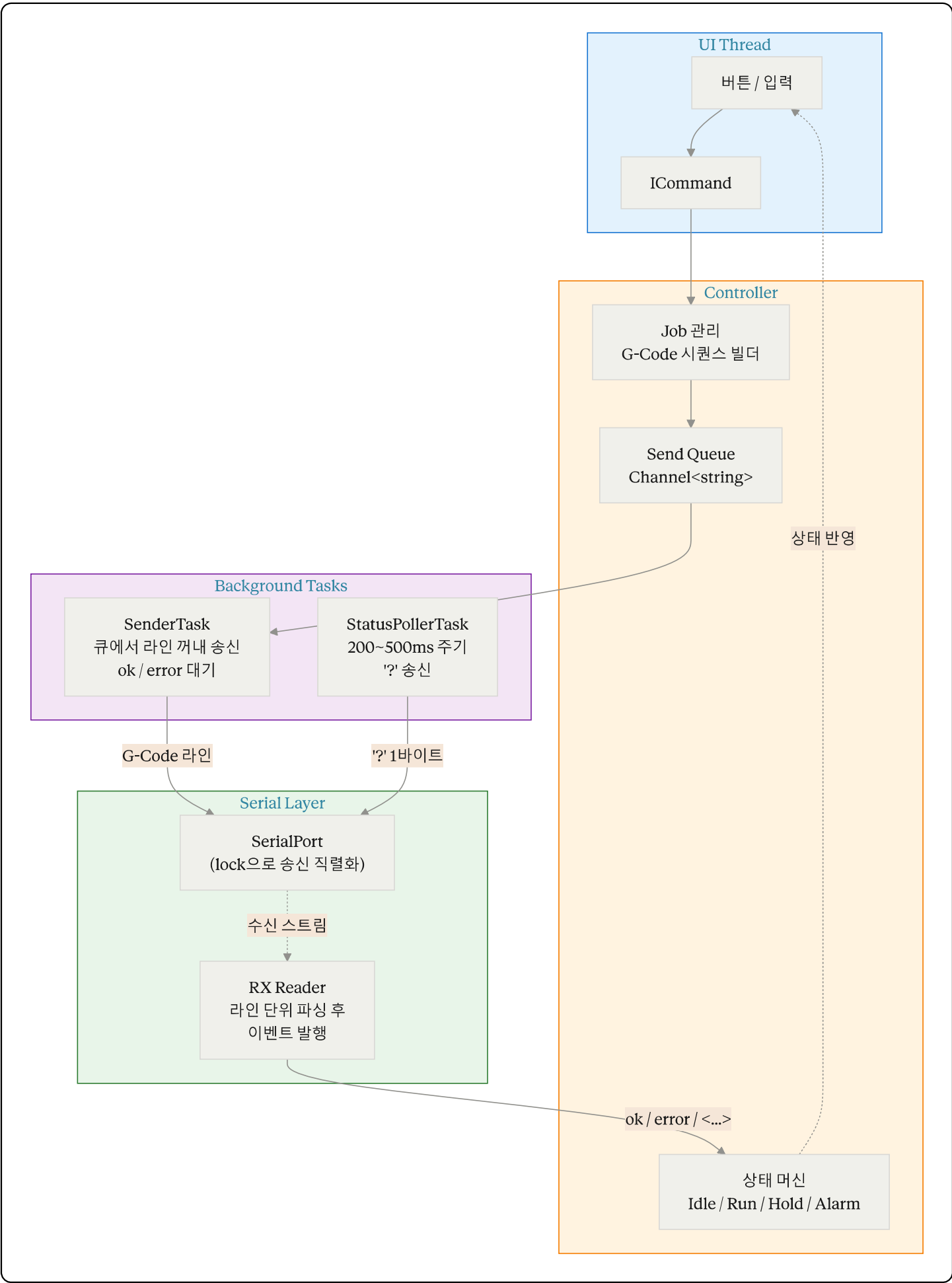
그 외 GRBL 코어 코드(파서, 플래너, 스테퍼)는 손대지 않는다. 손대면 유지보수가 매우 어려워진다.

7.C# 측 사용 기술 스택

7.1 필수 기술

영역	사용 라이브러리/기술	비고
시리얼 통신	<code>System.IO.Ports.SerialPort</code>	.NET 표준. NuGet <code>System.IO.Ports</code> 추가 필요 (.NET Core/5+ 환경)
비동기 처리	<code>async</code> , <code>await</code> , <code>Task</code> , <code>CancellationToken</code>	송신/수신/폴링 분리
스레드 안전 큐	<code>BlockingCollection<string></code> 또는 <code>Channel<string></code> (System.Threading.Channels)	G-Code 송신 큐
UI	WinForms 또는 WPF	둘 다 무방
상태/위치 파싱	정규식 (<code>System.Text.RegularExpressions</code>)	<code><...></code> 응답 파싱
로깅	Serilog 또는 NLog	송수신 로그는 운용에 매우 중요

7.2 권장 아키텍처



7.3 구현 시 주의사항

- **SerialPort의 `DataReceived` 이벤트는 라인 단위가 아니다.** 별도 RX 버퍼에 누적하면서 LF로 잘라야 한다. 또는 `BaseStream`에서 직접 비동기 read.
- **송신은 반드시 직렬화** — (?) 폴링과 G-Code 송신이 동시에 일어나지 않도록 lock 또는 단일 채널 사용.
- **(?) 응답은 `ok`가 아니다.** `<...>` 형식이며 `ok` 시퀀스와 무관하게 도착하므로 RX 파서에서 분리 처리.
- **라인 종단은 `\n`만.** `\r\n` 보내지 말 것 (GRBL은 무시하지만 일부 클론 보드에서 문제 발생).
- **Alarm 상태에서 모션 명령 거부됨** — UI에서 명확히 분리.
- **포트 끊김 감지** — `SerialPort.IsOpen` 만으로는 부족. 주기적 (?) 응답 타임아웃을 헬스체크로 활용.

7.4 참고할 만한 오픈소스 GRBL 컨트롤러

직접 코드를 가져다 쓰진 않더라도 프로토콜 처리 패턴 참고용:

- **Candle** (C++/Qt) — <https://github.com/Denvi/Candle> — 단순한 구조, RX 파싱 참고하기 좋음
- **gSender** (Electron/JS) — <https://github.com/Sienci-Labs/gsender> — 모던한 상태 머신 구현
- **Universal Gcode Sender (UGS)** (Java) — <https://github.com/winder/Universal-G-Code-Sender> — 표준 격, 페이싱 알고리즘 정석

8. 통합 테스트 체크리스트

#	항목	기대 결과	담당
1	USB 연결 시 부팅 메시지 수신	<code>Grbl 1.1h ...</code>	양측
2	(?) 송신 → 응답 수신	<code><Alarm ...></code> 또는 <code><Idle ...></code>	양측
3	(\$H) 송신 → 홈잉 동작	양 축 리미트 hit 후 pull-off, <code>ok</code>	양측
4	단일축 1mm Jog	정확히 1mm 이동	양측
5	임의 좌표 G0 이동	좌표 도달, MPos 일치	양측
6	M3/M5 동작	액추에이터 ON/OFF 확인	양측

#	항목	기대 결과	담당
7	편칭 시퀀스 10회 자동 실행	모든 좌표에서 편칭, 원점 복귀	양측
8	작업 중 ! 송신	즉시 감속 정지, Hold 상태	양측
9	~ 송신	정지 지점에서 재개	양측
10	\x18 (Ctrl-X)	즉시 리셋, 위치 유지	양측
11	작업 영역 밖 G-Code	error:2 또는 ALARM:2	양측
12	운전 중 리밋 강제 hit	ALARM:1, 정지	양측

9. 부록

9.1 본 문서가 다루지 않는 항목

- 멀티 헤드/다축 동기화
- 카메라 비전 정렬 / 좌표 보정
- 네트워크 통신 (TCP/IP, OPC UA 등)
- 데이터베이스 / 작업 이력 저장
- 사용자 인증 / 권한 관리

위 항목들은 별도 문서로 다룬다.

9.2 참고 문서

- GRBL Wiki: <https://github.com/gnea/grbl/wiki>
- GRBL Configuration: <https://github.com/gnea/grbl/wiki/Grbl-v1.1-Configuration>
- GRBL Interface: <https://github.com/gnea/grbl/wiki/Grbl-v1.1-Interface>
- GRBL Commands: <https://github.com/gnea/grbl/wiki/Grbl-v1.1-Commands>
- RAMPS 1.4 Pinout: https://reprap.org/wiki/RAMPS_1.4

9.3 변경 이력

버전	일자	작성자	내용
0.1	2026-04-29	-	초안 작성

